

OASIS INFOBYTE

DATA ANALYTICS INTERNSHIP

LEVEL 1 – TASK 4

SENTIMENT ANALYSIS

Final Project Report

Submitted By

Madala Sai Sprisha

Internship

OASIS INFOBYTE – Data Analytics Internship

Domain

Data Analytics / Machine Learning

Project Title

Level 2 – Task 1: Predicting House Prices with Linear Regression

Technologies Used

Python • Pandas • NumPy • Matplotlib • Scikit-learn • Jupyter Notebook

Submitted To

OASIS INFOBYTE

Year

2026

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to OASIS INFOBYTE for providing me with the opportunity to complete Level 2 – Task 1, Predicting House Prices with Linear Regression. This project provided practical exposure to exploratory data analysis, feature selection, preprocessing, train-test splitting, regression modelling, model evaluation, residual analysis, feature-impact interpretation, and regularized regression.

I also thank the mentors and coordinators for providing the internship task requirements and learning resources. The structured workflow helped me understand how a raw tabular dataset can be transformed into a suitable input for predictive modelling and how model performance can be assessed using appropriate evaluation metrics.

The implementation was completed in a Jupyter Notebook using the House Prices – Advanced Regression Techniques dataset. The project demonstrates a complete predictive modelling workflow, beginning with dataset inspection and ending with model comparison and final conclusions.

This work strengthened my practical understanding of supervised machine learning and demonstrated the importance of data preprocessing, meaningful feature selection, quantitative evaluation, and residual diagnostics when building a house-price prediction model.

ABSTRACT

This project focuses on predicting house sale prices using Linear Regression and comparing its performance with Ridge and Lasso Regression. The House Prices – Advanced Regression Techniques dataset contains 1,460 records and 81 columns, with SalePrice selected as the target variable. The workflow begins with dataset inspection, missing-value analysis, descriptive statistics, SalePrice distribution analysis, and correlation analysis.

Feature selection was guided by numerical correlations with SalePrice and domain relevance. OverallQual and GrLivArea showed the strongest numerical relationships with SalePrice. The Id field was excluded because it is an identifier rather than a meaningful property characteristic. Numerical missing values were handled using median imputation, categorical missing values using most-frequent imputation, and categorical variables were transformed using One-Hot Encoding.

The dataset was divided into 80% training data and 20% testing data, producing 1,168 training samples and 292 testing samples. Linear Regression achieved an MSE of 979,376,460.09, an RMSE of 31,294.99, and an R^2 score of 0.8723. Residual analysis showed that most errors were concentrated around zero, although larger errors occurred for some higher-priced houses.

As an extended comparison, Ridge and Lasso Regression were evaluated using the same train-test split and preprocessing approach. Lasso produced the best test performance with an RMSE of 28,304.21 and an R^2 score of 0.8956. The results demonstrate that regularization can improve predictive performance when a dataset contains many correlated and one-hot encoded features.

TABLE OF CONTENTS

CHAPTER	TITLE	PAGE NO
	Cover Page	1
	Acknowledgement	2
	Abstract	3
	Table of contents	4-5
Chapter 1	Introduction	
1.1	Project Overview	6
1.2	Problem Statement	6
1.3	Project Objectives	6
Chapter 2	Dataset & Tools	
2.1	Dataset Description	7
2.2	Tools & Technologies Used	7
2.3	Project Folder Structure	8
2.4	Input Dataset	8
Chapter 3	House Price Prediction Implementation	
3.1	Library Import & Dataset Loading	9
3.2	Data Inspection & Quality Assessment	10
3.3	Missing-Value Analysis	11
3.4	Descriptive Statistics	12
3.5	SalePrice Distribution	12
3.6	Correlation Analysis	13
3.7	Feature Selection Discussion	14
3.8	Data Preprocessing & One-Hot Encoding	15
3.9	Train-Test Split	17
3.10	Linear Regression Model	16
3.11	Model Evaluation	17
3.12	Actual vs Predicted Price Analysis	18

3.13	Residual Analysis	19
3.14	Residual Distribution	20
3.15	Feature Coefficient Analysis	20
3.16	Ridge & Lasso Regularization	21
3.17	Final Findings	22
Chapter 4	Results & Discussion	
4.1	Key Findings	23
4.2	Conclusion	23
4.3	Future Enhancements	24
4.4	References	24

CHAPTER 1: INTRODUCTION

1.1 Project Overview

The Predicting House Prices with Linear Regression project focuses on building a supervised machine-learning model that estimates house sale prices from property characteristics. The workflow begins with loading and inspecting the House Prices dataset, followed by missing-value analysis, descriptive statistics, distribution analysis, correlation analysis, feature selection, preprocessing, train-test splitting, model training, evaluation, residual diagnostics, and model comparison.

The implementation uses Python, Pandas and NumPy for data handling and numerical operations, Matplotlib for visualization, and Scikit-learn for preprocessing, regression modelling, and evaluation. The complete workflow is documented in a Jupyter Notebook.

1.2 Problem Statement

House-price datasets contain a mixture of numerical and categorical variables, missing observations, and features measured on different scales. A predictive model must therefore handle missing values and categorical attributes consistently before learning relationships between property characteristics and SalePrice. The project addresses this problem through a reproducible preprocessing pipeline and evaluates how well Linear Regression predicts unseen house prices.

1.3 Project Objectives

1. Load and inspect the House Prices dataset.
2. Perform missing-value analysis and descriptive statistical analysis.
3. Study the distribution of SalePrice and identify important numerical relationships.
4. Discuss relevant predictive features and exclude the Id identifier from modelling.
5. Handle numerical and categorical missing values using a preprocessing pipeline.
6. Convert categorical features using One-Hot Encoding.
7. Split the dataset into 80% training and 20% testing data.
8. Train and evaluate a Linear Regression model using MSE, RMSE, and R^2 .
9. Perform actual-vs-predicted and residual analysis.
10. Analyze feature coefficients and compare Linear Regression with Ridge and Lasso.

CHAPTER 2: DATASET & TOOLS

2.1 Dataset Description

The project uses the House Prices – Advanced Regression Techniques dataset. The executed notebook reports an initial shape of 1,460 rows and 81 columns. The dataset includes property characteristics such as MSSubClass, LotArea, Neighborhood, OverallQual, YearBuilt, TotalBsmtSF, GrLivArea, GarageCars, GarageArea, and many other attributes. SalePrice is used as the target variable.

2.2 Tools & Technologies Used

TOOL / TECHNOLOGY	PURPOSE
Python	Programming language and machine-learning workflow
Pandas	Dataset loading, inspection, transformation, and tabular analysis
NumPy	Numerical operations and supporting calculations
Matplotlib	Plots and visual analysis
Scikit-learn	Preprocessing, Linear Regression, Ridge, Lasso, and evaluation metrics
Jupyter Notebook	Interactive implementation and documentation
CSV	Input dataset storage

2.3 Project Folder Structure

The project is organized into separate folders and files for the input dataset, implementation notebook, generated outputs, screenshots, and report documentation. This structure keeps the project components organized and makes the implementation and supporting evidence easy to access. The submitted notebook follows a structured workflow covering dataset loading, data inspection, preprocessing, model development, evaluation, diagnostics, and model comparison.

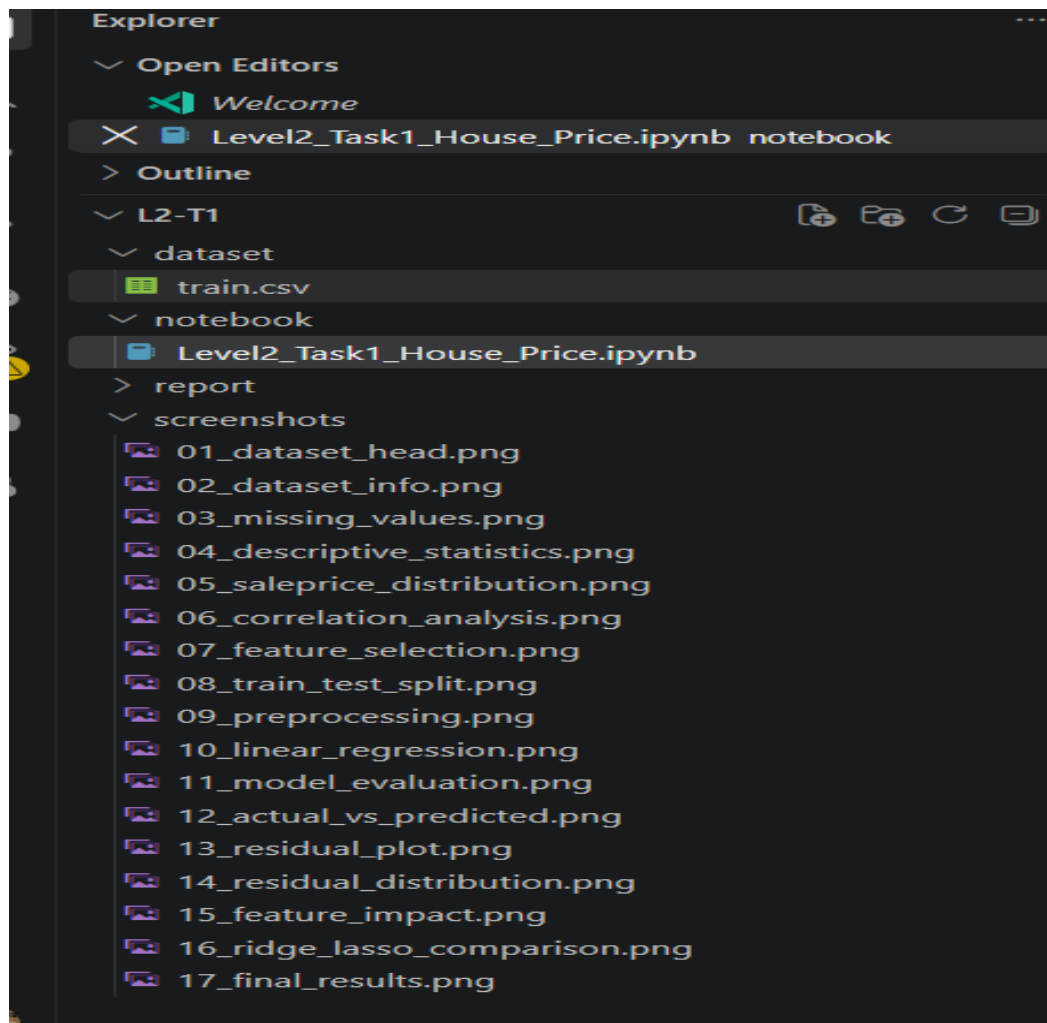


Figure 2.3: Project Folder Structure of the House Price Prediction Project

2.4 Input Dataset

The project uses the House Prices – Advanced Regression Techniques dataset for predicting residential property prices. The dataset contains property-related features describing aspects such as house characteristics, area, quality, and other attributes that can influence the final sale price. The SalePrice variable is used as the target variable for the regression model, while the remaining relevant features are considered as input variables.

The dataset is loaded into Python using Pandas and inspected for its structure, data types, missing values, descriptive statistics, and relationships with the target variable before preprocessing and model training.

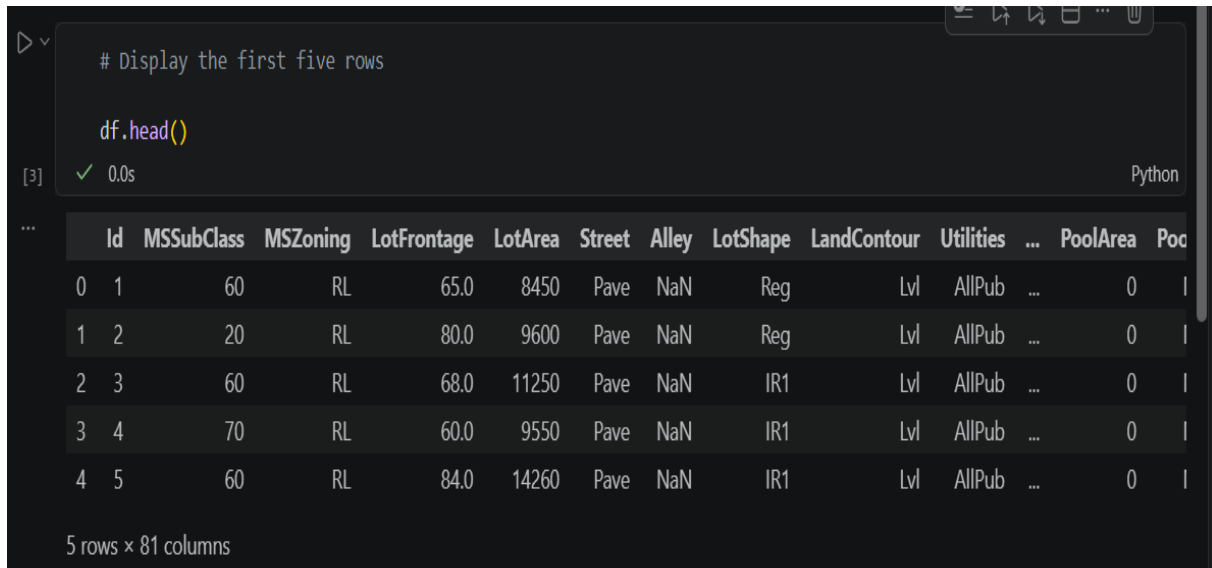
[illegible]

Figure 2.4: Input Dataset – House Prices Dataset Used for Model Development

CHAPTER 3: IMPLEMENTATION

3.1 Dataset Loading

The House Prices CSV file was loaded into a Pandas DataFrame using `pd.read_csv()`. The notebook confirmed successful loading and reported a dataset shape of 1,460 rows and 81 columns.



```
# Display the first five rows

df.head()
```

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape	LandContour	Utilities	...	PoolArea	PoolCount
0	1	60	RL	65.0	8450	Pave	NaN	Reg	Lvl	AllPub	...	0	1
1	2	20	RL	80.0	9600	Pave	NaN	Reg	Lvl	AllPub	...	0	1
2	3	60	RL	68.0	11250	Pave	NaN	IR1	Lvl	AllPub	...	0	1
3	4	70	RL	60.0	9550	Pave	NaN	IR1	Lvl	AllPub	...	0	1
4	5	60	RL	84.0	14260	Pave	NaN	IR1	Lvl	AllPub	...	0	1

5 rows x 81 columns

Figure 3.1: Initial Dataset Preview

Interpretation:

The first five records confirm that the House Prices dataset contains property-level observations with numerical and categorical attributes. SalePrice is the target variable, while Id is an identifier and is excluded from the predictive feature set.

3.2 Dataset Information

The DataFrame information output was used to inspect the number of columns, non-null counts, and data types. The dataset contains numerical variables represented mainly as integer and floating-point types and categorical variables represented as object types.

```
# Display dataset information

df.info()

[4] ✓ 0.0s

... 53 KitchenQual    1460 non-null    object
    54 TotRmsAbvGrd    1460 non-null    int64
    55 Functional      1460 non-null    object
    56 Fireplaces      1460 non-null    int64
    57 FireplaceQu     770 non-null    object
    58 GarageType       1379 non-null    object
    59 GarageYrBlt      1379 non-null    float64
    60 GarageFinish    1379 non-null    object
    61 GarageCars       1460 non-null    int64
    62 GarageArea       1460 non-null    int64
    63 GarageQual       1379 non-null    object
    64 GarageCond       1379 non-null    object
    65 PavedDrive       1460 non-null    object
    66 WoodDeckSF       1460 non-null    int64
    67 OpenPorchSF      1460 non-null    int64
    68 EnclosedPorch     1460 non-null    int64
    69 3SsnPorch        1460 non-null    int64
    70 ScreenPorch      1460 non-null    int64
    71 PoolArea         1460 non-null    int64
    72 PoolQC           7 non-null    object
    73 Fence            281 non-null    object
    74 MiscFeature       54 non-null    object
    75 MiscVal          1460 non-null    int64
    76 MoSold           1460 non-null    int64
    77 YrSold           1460 non-null    int64
    78 SaleType          1460 non-null    object
    79 SaleCondition     1460 non-null    object
    80 SalePrice        1460 non-null    int64
dtypes: float64(3), int64(35), object(43)
memory usage: 924.0+ KB
```

Figure 3.2: Dataset Information and Data Types

Interpretation:

The dataset contains 1,460 records and 81 columns. The information output shows a mixture of integer, floating-point, and categorical object variables, indicating that both numerical and categorical preprocessing are required.

3.3 Missing-Value Analysis

Missing-value analysis identified 19 columns with at least one missing observation. The largest missing counts were PoolQC (1,453), MiscFeature (1,406), Alley (1,369), Fence (1,179), MasVnrType (872), and FireplaceQu (690). Other variables contained smaller numbers of missing observations, including LotFrontage, garage-related variables, basement variables, MasVnrArea, and Electrical.

```
# Missing-value analysis

missing_values = df.isnull().sum().sort_values(ascending=False)

missing_values = missing_values[missing_values > 0]

print("Columns with missing values:")
display(missing_values.to_frame(name="Missing Count"))
```

[5] ✓ 0.0s Pyt

... Columns with missing values:

...

	Missing Count
PoolQC	1453
MiscFeature	1406
Alley	1369
Fence	1179
MasVnrType	872
FireplaceQu	690
LotFrontage	259
GarageQual	81
GarageFinish	81
GarageType	81
GarageYrBlt	81
GarageCond	81
BsmtFinType2	38
BsmtExposure	38

Figure 3.3: Missing-Value Analysis

Interpretation:

Several variables contain missing observations. The largest missing counts occur in PoolQC, MiscFeature, Alley, Fence, and MasVnrType. The analysis supports the use of a systematic preprocessing pipeline rather than manual row deletion.

3.4 Descriptive Statistics

Descriptive statistics were generated for the numerical variables. For SalePrice, the notebook reports a mean of 180,921.20, a standard deviation of 79,442.50, a minimum of 34,900, a first quartile of 129,975, a median of 163,000, and a maximum of 755,000. These values provide an initial understanding of the target variable's scale and variability.

```
# Descriptive statistics
display(df.describe().T)
```

	count	mean	std	min	25%	50%
Id	1460.0	730.500000	421.610009	1.0	365.75	730.5
MSSubClass	1460.0	56.897260	42.300571	20.0	20.00	50.0
LotFrontage	1201.0	70.049958	24.284752	21.0	59.00	69.0
LotArea	1460.0	10516.828082	9981.264932	1300.0	7553.50	9478.5
OverallQual	1460.0	6.099315	1.382997	1.0	5.00	6.0
OverallCond	1460.0	5.575342	1.112799	1.0	5.00	5.0
YearBuilt	1460.0	1971.267808	30.202904	1872.0	1954.00	1973.0
YearRemodAdd	1460.0	1984.865753	20.645407	1950.0	1967.00	1994.0
MasVnrArea	1452.0	103.685262	181.066207	0.0	0.00	0.0
BsmtFinSF1	1460.0	443.639726	456.098091	0.0	0.00	383.5
BsmtFinSF2	1460.0	46.549315	161.319273	0.0	0.00	0.0
BsmtUnfSF	1460.0	567.240411	441.866955	0.0	223.00	477.5
TotalBsmtSF	1460.0	1057.429452	438.705324	0.0	795.75	991.5
1stFlrSF	1460.0	1162.626712	386.587738	334.0	882.00	1087.0
2ndFlrSF	1460.0	346.992466	436.528436	0.0	0.00	0.0
LowQualFinSF	1460.0	5.844521	48.623081	0.0	0.00	0.0
GrLivArea	1460.0	1515.463699	525.480383	334.0	1129.50	1464.0
BsmtFullBath	1460.0	0.425342	0.518911	0.0	0.00	0.0
BsmtHalfBath	1460.0	0.057534	0.238753	0.0	0.00	0.0

Figure 3.4: Descriptive Statistics

Interpretation:

The descriptive statistics summarize the numerical variables through count, mean, standard deviation, quartiles, minimum, and maximum values. SalePrice has a mean of approximately 180,921 and a median of 163,000, indicating a positively skewed price distribution.

3.5 SalePrice Distribution

A histogram was created to visualize the distribution of house sale prices. Most observations fall within the lower-to-middle portion of the price range, while a smaller number of expensive properties extend the distribution to the right.

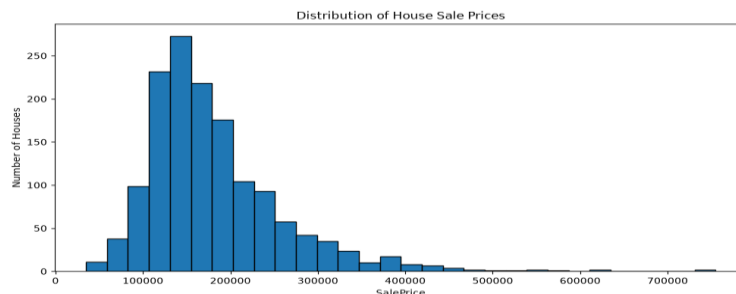
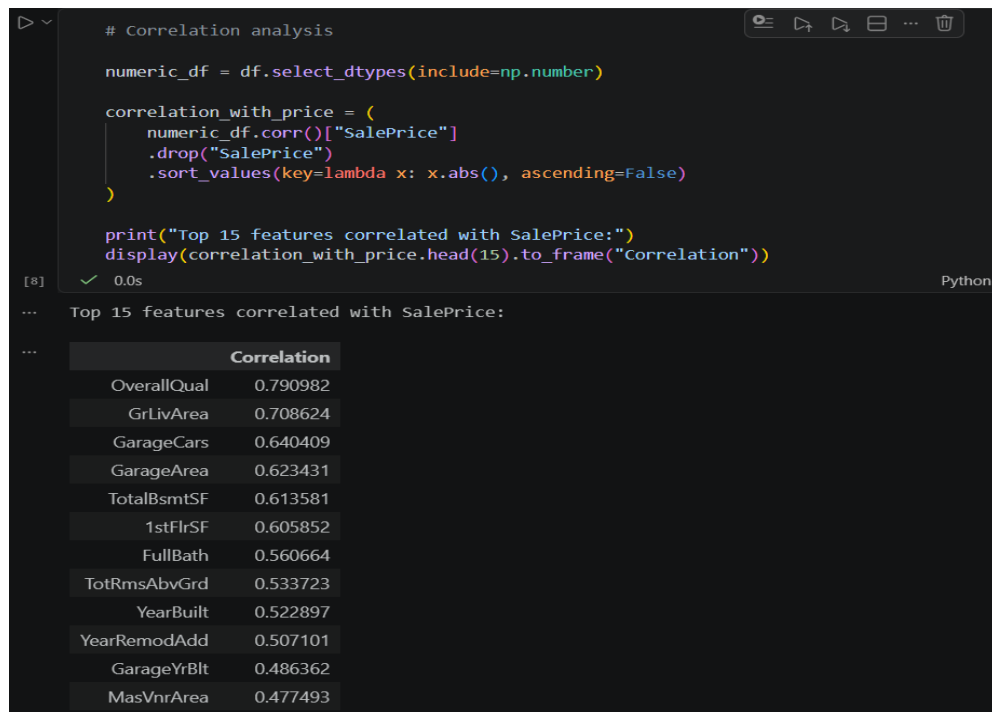


Figure 3.5: Distribution of House Sale Prices

Interpretation: The SalePrice histogram is concentrated in the lower-to-middle price range and has a long right tail. This indicates that relatively expensive properties are less frequent and that prediction errors may be larger for some high-priced houses.

3.6 Correlation Analysis

Correlation analysis was performed on numerical variables by calculating their Pearson correlation with SalePrice and ranking the results by absolute correlation. OverallQual has the strongest correlation at 0.790982, followed by GrLivArea at 0.708624. GarageCars, GarageArea, TotalBsmtSF, 1stFlrSF, FullBath, TotRmsAbvGrd, YearBuilt, and YearRemodAdd also show positive relationships with SalePrice.



```
# Correlation analysis

numeric_df = df.select_dtypes(include=np.number)

correlation_with_price = (
    numeric_df.corr()["SalePrice"]
    .drop("SalePrice")
    .sort_values(key=lambda x: x.abs(), ascending=False)
)

print("Top 15 features correlated with SalePrice:")
display(correlation_with_price.head(15).to_frame("Correlation"))
```

[8] ✓ 0.0s Python

Top 15 features correlated with SalePrice:

	Correlation
OverallQual	0.790982
GrLivArea	0.708624
GarageCars	0.640409
GarageArea	0.623431
TotalBsmtSF	0.613581
1stFlrSF	0.605852
FullBath	0.560664
TotRmsAbvGrd	0.533723
YearBuilt	0.522897
YearRemodAdd	0.507101
GarageYrBlt	0.486362
MasVnrArea	0.477493

Figure 3.6: Correlation of Numerical Features with SalePrice

Interpretation:

OverallQual has the strongest numerical correlation with SalePrice (0.790982), followed by GrLivArea (0.708624), GarageCars (0.640409), GarageArea (0.623431), and TotalBsmtSF (0.613581). These results guided the feature-selection discussion.

3.7 Feature Selection Discussion

The notebook identifies OverallQual, GrLivArea, GarageCars, GarageArea, TotalBsmtSF, 1stFlrSF, FullBath, TotRmsAbvGrd, YearBuilt, and YearRemodAdd as important numerical features based on correlation and domain relevance. OverallQual is the strongest numerical predictor, followed by GrLivArea. Categorical features such as Neighborhood are also considered useful because location can strongly influence property prices. The Id column is excluded because it is only an identifier.

```
## Feature Selection Discussion

The correlation analysis shows that OverallQual, GrLivArea, GarageCars,
GarageArea, TotalBsmtSF, 1stFlrSF, FullBath, TotRmsAbvGrd, YearBuilt,
and YearRemodAdd have strong positive relationships with SalePrice.

OverallQual is the strongest numerical predictor, followed by GrLivArea.
These features are likely to be important because house quality, living
space, garage capacity, basement area, number of rooms, and construction
or renovation year can influence property value.

Categorical features such as Neighborhood are also expected to be useful
because location can strongly influence house prices.

The Id column will be excluded because it is only an identifier and does
not represent a meaningful house characteristic.
```

markdown

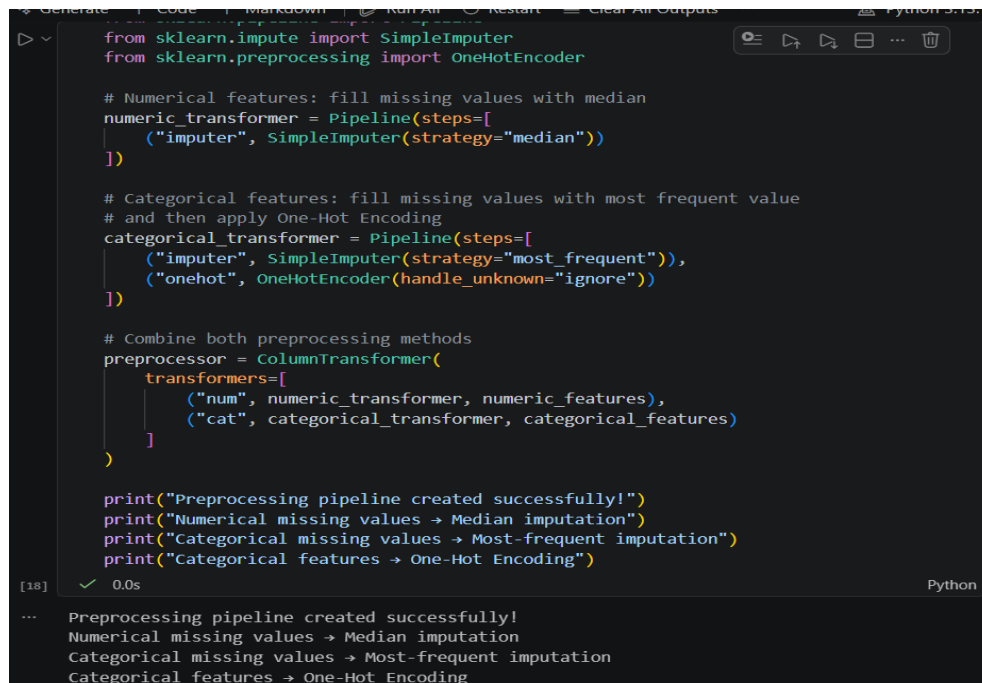
Figure 3.7: Feature Selection Discussion

Interpretation:

The feature-selection discussion identifies property quality, living area, garage capacity, basement area, room count, and construction or renovation year as potentially important predictors. Neighborhood is also expected to be useful because location can strongly influence house prices. The Id column is excluded because it is only an identifier.

3.8 Preprocessing

The modelling features were created by removing SalePrice from the predictors and excluding Id. The notebook reports 36 numerical features and 43 categorical features before encoding, for a total of 79 input features. Numerical variables were processed with median imputation, while categorical variables were processed with most-frequent imputation and One-Hot Encoding. A ColumnTransformer combines these transformations into a single reproducible preprocessing pipeline.



```
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import OneHotEncoder

# Numerical features: fill missing values with median
numeric_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="median"))
])

# Categorical features: fill missing values with most frequent value
# and then apply One-Hot Encoding
categorical_transformer = Pipeline(steps=[
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("onehot", OneHotEncoder(handle_unknown="ignore"))
])

# Combine both preprocessing methods
preprocessor = ColumnTransformer(
    transformers=[
        ("num", numeric_transformer, numeric_features),
        ("cat", categorical_transformer, categorical_features)
    ]
)

print("Preprocessing pipeline created successfully!")
print("Numerical missing values → Median imputation")
print("Categorical missing values → Most-frequent imputation")
print("Categorical features → One-Hot Encoding")
```

[18] ✓ 0.0s Python

Preprocessing pipeline created successfully!
Numerical missing values → Median imputation
Categorical missing values → Most-frequent imputation
Categorical features → One-Hot Encoding

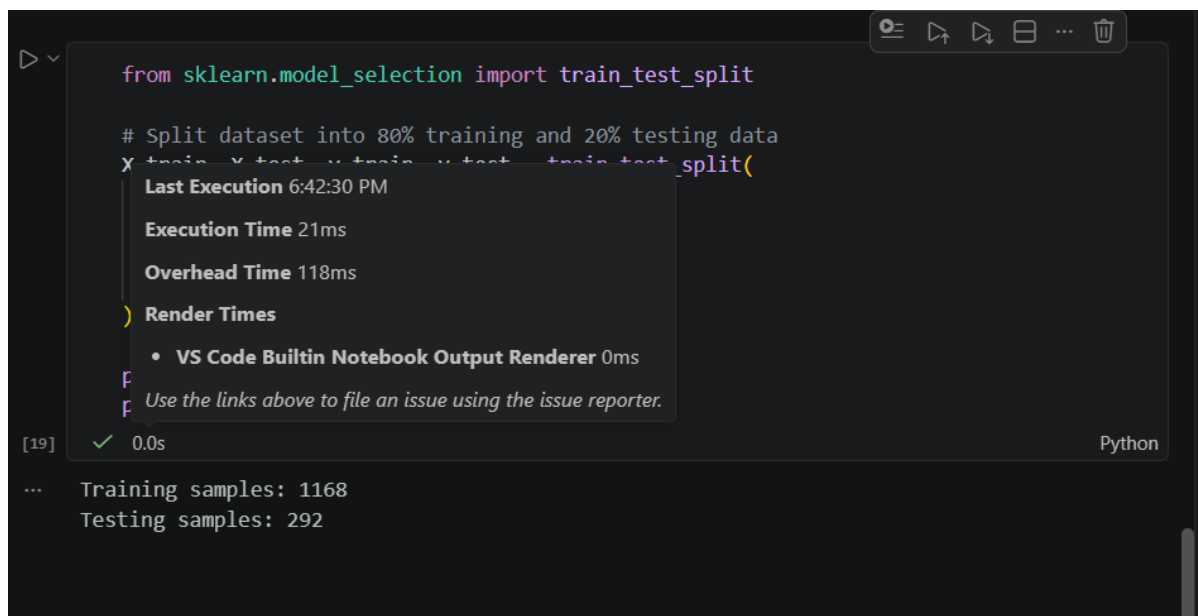
Figure 3.9: Preprocessing Pipeline

Interpretation:

Numerical missing values are handled using median imputation. Categorical missing values are handled using the most-frequent strategy, followed by One-Hot Encoding. The ColumnTransformer combines both transformations so that preprocessing is applied consistently during model training and prediction.

3.9 Train-Test Split

The prepared dataset was divided into training and testing subsets using `train_test_split()` with `test_size=0.20` and `random_state=42`. The resulting split contains 1,168 training samples and 292 testing samples.



```
from sklearn.model_selection import train_test_split

# Split dataset into 80% training and 20% testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

[19] ✓ 0.0s Python
```

Training samples: 1168
Testing samples: 292

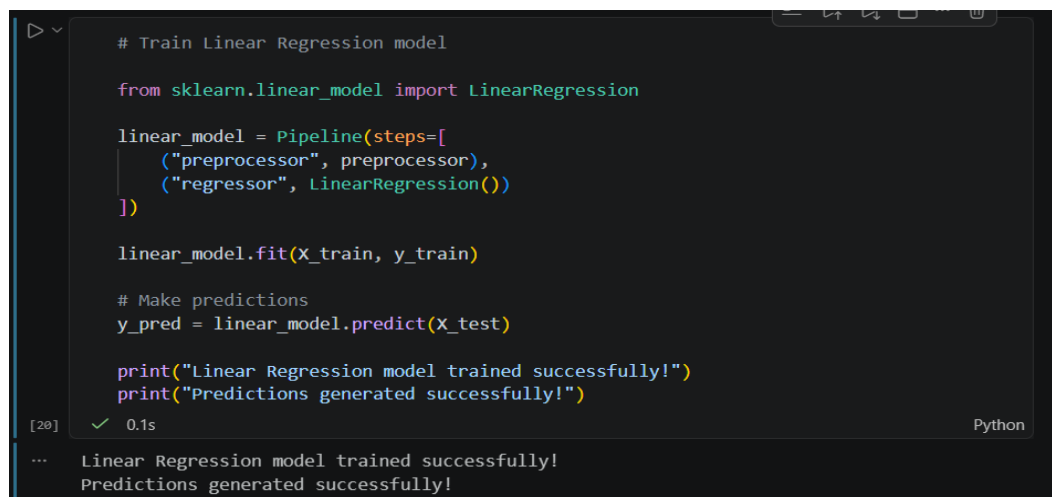
Figure 3.8: Training and Testing Split

Interpretation:

The dataset was divided using an 80:20 train-test split with `random_state=42`. This produced 1,168 training samples and 292 testing samples, providing separate data for model fitting and unbiased performance evaluation.

3.10 Linear Regression

Linear Regression was implemented in a Scikit-learn Pipeline that first applies the preprocessing transformer and then fits the LinearRegression estimator. The model was trained on the 1,168 training observations and used to generate predictions for the 292 testing observations.



```
# Train Linear Regression model

from sklearn.linear_model import LinearRegression

linear_model = Pipeline(steps=[
    ("preprocessor", preprocessor),
    ("regressor", LinearRegression())
])

linear_model.fit(X_train, y_train)

# Make predictions
y_pred = linear_model.predict(X_test)

print("Linear Regression model trained successfully!")
print("Predictions generated successfully!")

[20] ✓ 0.1s Python
```

Linear Regression model trained successfully!
Predictions generated successfully!

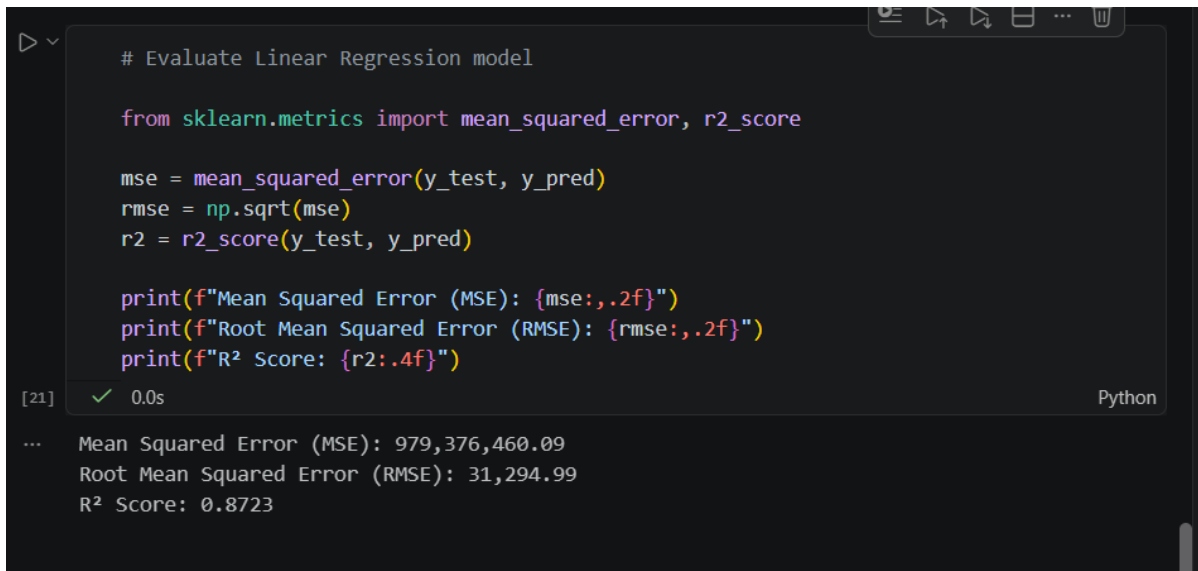
Figure 3.10: Linear Regression Model Training

Interpretation:

The Linear Regression model was implemented as a pipeline containing the preprocessing transformer and the regression estimator. The model trained successfully and generated predictions for the test set.

3.11 Model Evaluation

The baseline Linear Regression model was evaluated using Mean Squared Error (MSE), Root Mean Squared Error (RMSE), and R^2 . The obtained results were $MSE = 979,376,460.09$, $RMSE = 31,294.99$, and $R^2 = 0.8723$.



```
# Evaluate Linear Regression model

from sklearn.metrics import mean_squared_error, r2_score

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f"Mean Squared Error (MSE): {mse:,.2f}")
print(f"Root Mean Squared Error (RMSE): {rmse:,.2f}")
print(f"R2 Score: {r2:.4f}")
```

[21] ✓ 0.0s Python

... Mean Squared Error (MSE): 979,376,460.09
Root Mean Squared Error (RMSE): 31,294.99
R² Score: 0.8723

Figure 3.11: Linear Regression Model Evaluation

Interpretation:

The baseline Linear Regression model achieved an MSE of 979,376,460.09, an RMSE of 31,294.99, and an R^2 score of 0.8723. The R^2 value indicates that the model explains approximately 87.23% of the variation in SalePrice on the test data.

3.12 Actual vs Predicted House Prices

The actual-versus-predicted scatter plot compares the observed SalePrice values with the values predicted by Linear Regression. The dashed diagonal line represents perfect prediction. Most points remain reasonably close to this line, while larger deviations are visible for some higher-priced houses.

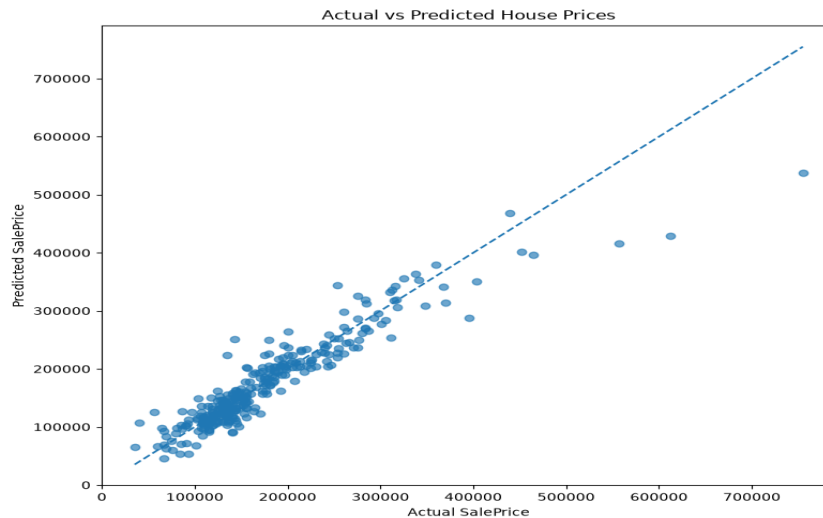


Figure 3.12: Actual vs Predicted House Prices

Interpretation:

Most observations are distributed around the diagonal reference line, showing a strong overall relationship between actual and predicted prices. Some higher-priced houses deviate substantially from the reference line, indicating larger prediction errors for a subset of expensive properties.

3.13 Residual Analysis

Residuals were calculated as Actual SalePrice minus Predicted SalePrice. The residual-versus-predicted plot uses a horizontal zero reference line to assess whether prediction errors are centered around zero. Most observations are close to the reference line, but the spread increases for larger predicted prices and several influential residuals are visible.

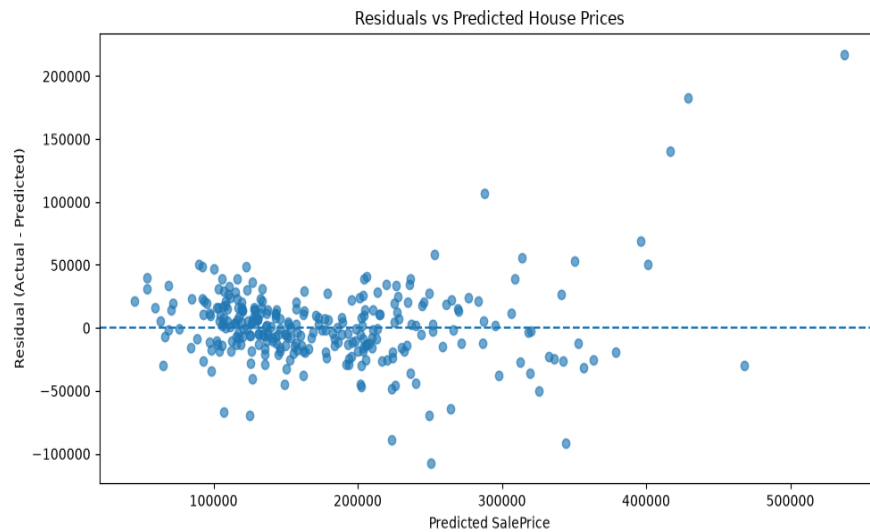


Figure 3.13: Residuals vs Predicted House Prices

Interpretation:

Most residuals are centered around the zero line, which indicates that predictions are reasonably balanced for many observations. The spread becomes wider for higher predicted prices and several large positive and negative residuals are visible, suggesting less consistent performance for expensive properties.

3.14 Residual Distribution

The histogram of Linear Regression residuals shows that the largest concentration of errors lies near zero. However, the distribution has extended tails, including several large positive residuals. This indicates that the model performs well for many properties but can underpredict or overpredict certain observations by a substantial amount.

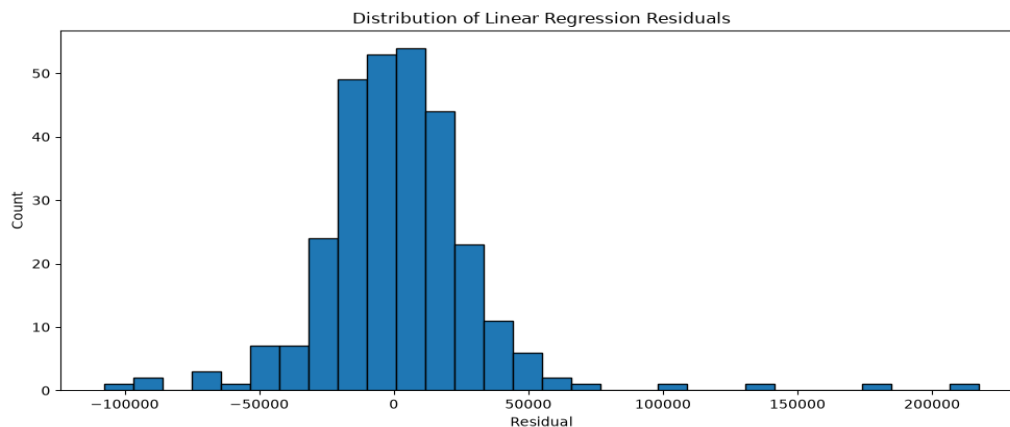


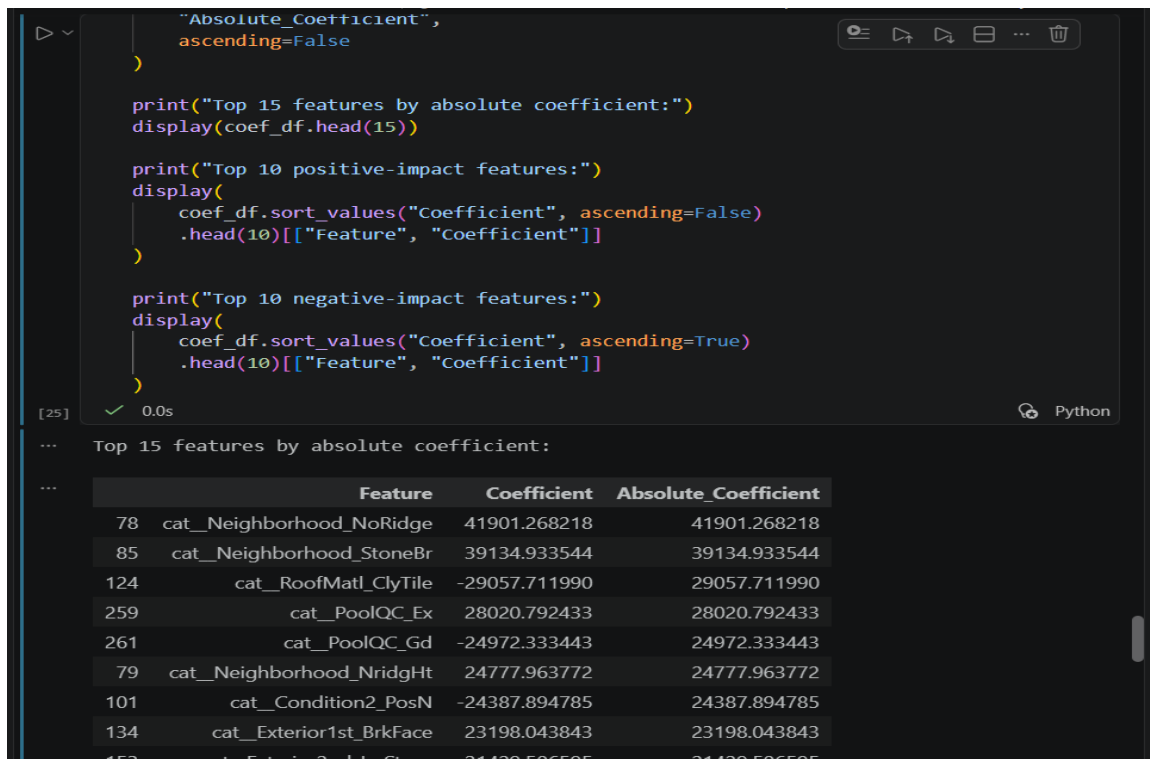
Figure 3.14: Distribution of Linear Regression Residuals

Interpretation:

The residual distribution is concentrated around zero, while a smaller number of observations form long tails on both sides. The positive tail contains several large residuals, reflecting cases where actual prices substantially exceeded model predictions.

3.15 Feature Impact Analysis

Feature impact was examined using the coefficients of the fitted Linear Regression model after preprocessing and One-Hot Encoding. The largest absolute coefficients include encoded Neighborhood, RoofMatl, PoolQC, Condition2, Exterior, LandContour, BsmtQual, and KitchenQual categories. Neighborhood_NoRidge and Neighborhood_StoneBr have the largest positive coefficients in the displayed results, while RoofMatl_ClyTile has the largest negative coefficient. These coefficients describe the model's learned contribution within the encoded feature representation.



```
"Absolute_Coefficient",
ascending=False
)

print("Top 15 features by absolute coefficient:")
display(coef_df.head(15))

print("Top 10 positive-impact features:")
display(
    coef_df.sort_values("Coefficient", ascending=False)
    .head(10)[["Feature", "Coefficient"]]
)

print("Top 10 negative-impact features:")
display(
    coef_df.sort_values("Coefficient", ascending=True)
    .head(10)[["Feature", "Coefficient"]]
)

[25] ✓ 0.0s Python
```

	Feature	Coefficient	Absolute_Coefficient
78	cat_Neighborhood_NoRidge	41901.268218	41901.268218
85	cat_Neighborhood_StoneBr	39134.933544	39134.933544
124	cat_RoofMatl_ClyTile	-29057.711990	29057.711990
259	cat_PoolQC_Ex	28020.792433	28020.792433
261	cat_PoolQC_Gd	-24972.333443	24972.333443
79	cat_Neighborhood_NridgHt	24777.963772	24777.963772
101	cat_Condition2_PosN	-24387.894785	24387.894785
134	cat_Exterior1st_BrkFace	23198.043843	23198.043843
153	cat_Exterior2nd_ImStucc	21429.506595	21429.506595

Figure 3.15: Feature Impact Based on Linear Regression Coefficients

Interpretation:

The coefficient analysis shows that several encoded categorical features have large positive or negative coefficients. Neighborhood_NoRidge and Neighborhood_StoneBr have the largest positive coefficients in the displayed results, while RoofMatl_ClyTile has the largest negative coefficient. Coefficients indicate model-assigned effects within the encoded feature representation.

3.16 Ridge and Lasso Comparison

As an extension, Ridge Regression with $\alpha=10$ and Lasso Regression with $\alpha=100$ were trained using the same preprocessing pipeline and train-test split. The comparison shows that Lasso produced the lowest MSE and RMSE and the highest R^2 among the three evaluated models.

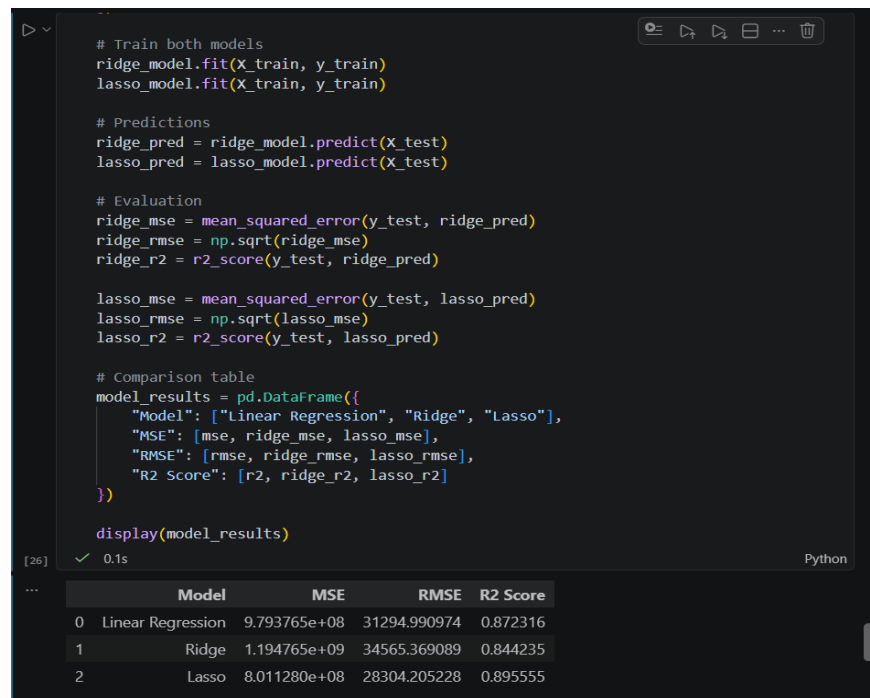


Figure 3.16: Comparison of Linear Regression, Ridge, and Lasso

Interpretation:

Lasso achieved the strongest test performance among the three evaluated models, with RMSE 28,304.21 and R^2 0.8956. Linear Regression achieved RMSE 31,294.99 and R^2 0.8723, while Ridge achieved RMSE 34,565.37 and R^2 0.8442. In this implementation, Lasso therefore provided the best test-set performance.

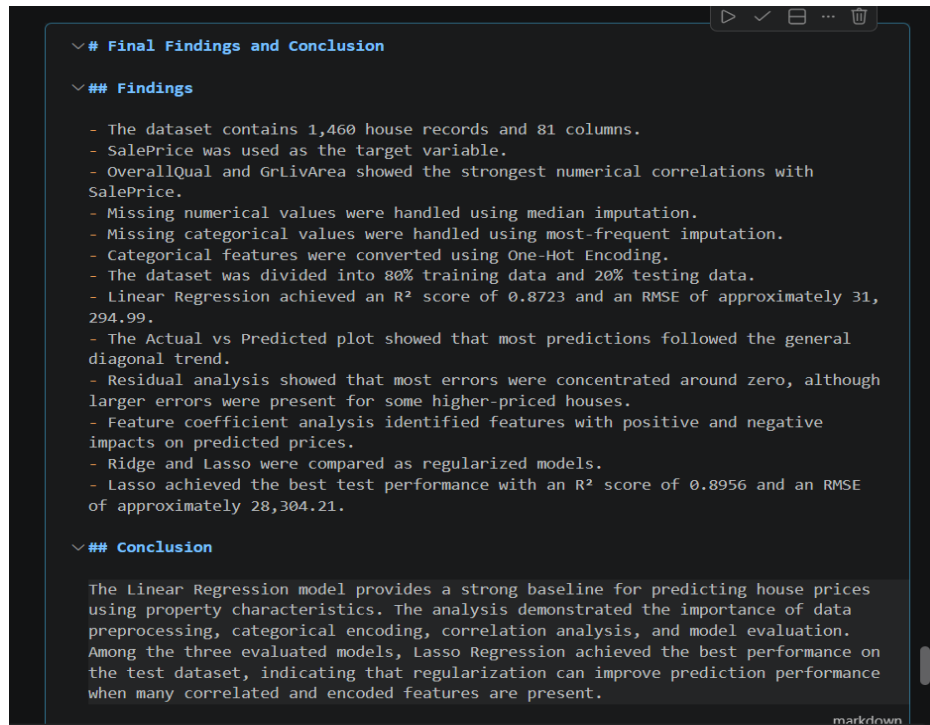
3.17 Final Findings

The final notebook findings summarize the complete implementation. The dataset contains 1,460 house records and 81 columns, with SalePrice as the target. OverallQual and GrLivArea showed the strongest numerical correlations with SalePrice. Missing numerical and categorical values were handled through the preprocessing pipeline, and categorical features were converted using One-Hot Encoding.

Linear Regression achieved an R^2 score of 0.8723 and an RMSE of approximately 31,294.99. The actual-versus-predicted plot showed a generally strong diagonal trend, while residual analysis indicated

that most errors were concentrated around zero with larger errors for some higher-priced houses. Feature coefficient analysis identified both positive- and negative-impact encoded features.

The regularized-model comparison showed that Lasso achieved the best test performance, with an R^2 score of 0.8956 and an RMSE of approximately 28,304.21. This result indicates that regularization can improve prediction performance when many correlated and encoded features are present.

A screenshot of a Jupyter Notebook interface. The notebook has a dark theme. The top section is titled "# Final Findings and Conclusion" and is expanded. Below it is a section titled "## Findings" which contains a bulleted list of 12 items summarizing the project workflow and results. The final item states that Lasso achieved the best performance with an R^2 score of 0.8956 and an RMSE of approximately 28,304.21. Below the findings is a section titled "## Conclusion" which provides a summary of the project, stating that Lasso Regression achieved the best performance among the tested alternatives. The notebook interface includes standard Jupyter controls like run, save, and delete buttons at the top right of the cell.

```
▼ # Final Findings and Conclusion
▼ ## Findings
- The dataset contains 1,460 house records and 81 columns.
- SalePrice was used as the target variable.
- OverallQual and GrLivArea showed the strongest numerical correlations with SalePrice.
- Missing numerical values were handled using median imputation.
- Missing categorical values were handled using most-frequent imputation.
- Categorical features were converted using One-Hot Encoding.
- The dataset was divided into 80% training data and 20% testing data.
- Linear Regression achieved an  $R^2$  score of 0.8723 and an RMSE of approximately 31,294.99.
- The Actual vs Predicted plot showed that most predictions followed the general diagonal trend.
- Residual analysis showed that most errors were concentrated around zero, although larger errors were present for some higher-priced houses.
- Feature coefficient analysis identified features with positive and negative impacts on predicted prices.
- Ridge and Lasso were compared as regularized models.
- Lasso achieved the best test performance with an  $R^2$  score of 0.8956 and an RMSE of approximately 28,304.21.

▼ ## Conclusion
The Linear Regression model provides a strong baseline for predicting house prices using property characteristics. The analysis demonstrated the importance of data preprocessing, categorical encoding, correlation analysis, and model evaluation. Among the three evaluated models, Lasso Regression achieved the best performance on the test dataset, indicating that regularization can improve prediction performance when many correlated and encoded features are present.
```

Figure 3.17: Final Findings and Conclusion

Interpretation:

The final findings consolidate the complete workflow: dataset inspection, correlation analysis, preprocessing, train-test splitting, Linear Regression evaluation, residual analysis, feature-impact analysis, and comparison with regularized models. The final conclusion identifies Lasso as the best-performing model among the tested alternatives.

CHAPTER 4: RESULTS AND DISCUSSION

4.1 Key Findings

METRIC	RESULT
Initial dataset	1,460 rows × 81 columns
Target variable	SalePrice
Numerical features before encoding	36
Categorical features before encoding	43
Training / testing split	1,168 / 292 samples
Linear Regression MSE	979,376,460.09
Linear Regression RMSE	31,294.99
Linear Regression R ²	0.8723
Ridge RMSE / R ²	34,565.37 / 0.8442
Lasso RMSE / R ²	28,304.21 / 0.8956
Best test model	Lasso Regression
Strongest numerical correlation	OverallQual = 0.790982
Second strongest numerical correlation	GrLivArea = 0.708624

The results demonstrate that the project completed the required predictive modelling workflow. The baseline Linear Regression model provides a strong benchmark, while Lasso improves the test-set performance among the evaluated models. The diagnostics also show that prediction errors are not uniform across the entire price range, particularly for some higher-priced properties.

4.2 Conclusion

The Level 2 – Task 1 Predicting House Prices with Linear Regression project was successfully completed using Python and Scikit-learn. The workflow systematically inspected the House Prices dataset, analyzed missing values and descriptive statistics, studied SalePrice distribution and correlations, selected relevant features, applied numerical and categorical preprocessing, split the data

into training and testing subsets, trained a Linear Regression model, evaluated its performance, and analyzed residuals and feature coefficients.

Linear Regression achieved an R^2 of 0.8723 with an RMSE of 31,294.99, providing a strong baseline for house-price prediction. The comparison with Ridge and Lasso showed that Lasso achieved the best test performance with an R^2 of 0.8956 and an RMSE of 28,304.21. Therefore, within this implementation and test split, Lasso Regression is the preferred model among the evaluated alternatives.

4.3 Future Enhancements

1. Apply feature engineering to capture nonlinear relationships among property characteristics.
2. Experiment with additional regularization parameters using cross-validation.
3. Investigate log transformation of SalePrice to address its right-skewed distribution.
4. Evaluate additional regression models and ensemble methods for comparison.
5. Use cross-validation and hyperparameter tuning to obtain more robust performance estimates.
6. Perform deeper error analysis on high-priced properties and influential observations.
7. Develop an interactive dashboard or deployment interface for house-price prediction.

4.4 References

1. House Prices – Advanced Regression Techniques dataset and the submitted Level 2 – Task 1 implementation notebook.
2. Scikit-learn functionality used for train-test splitting, preprocessing, Linear Regression, Ridge Regression, Lasso Regression, Mean Squared Error, Root Mean Squared Error, and R^2 evaluation.
3. Python, Pandas, NumPy, Matplotlib, and Jupyter Notebook used for data preparation, analysis, visualization, and documentation.